Comparative Analysis of Machine Learning and Deep Learning Approaches for Multi-Class Question-Answer Text Classificatnion

AVISHEK SENGUPTA
Department of Computer Science
Brac University
Dhaka, Bangladesh
1

MD ASHIKUZZAMAN
Department of Computer Science
Brac University
Dhaka, Bangladesh
2

FARDIN SAUROV
Department of Computer Science
Brac University
Dhaka, Bangladesh
3

***Abstract*— Question–answer (QA) classification is a key task in Natural Language Processing (NLP) that focuses on automatically organizing question–answer pairs into predefined categories. This capability is widely used in real-world applications such as chatbots, automated customer support systems, and information retrieval platforms. The project is a multi-class text classification task in which we train, test, and assess the dataset using various NLP pipelines that combine EDA, word representation, preprocessing, and classification models. In order to identify the best models that can predict the testing data with the highest accuracy/score and the fewest errors, we use various preprocessing techniques before passing it to baseline machine learning and neural network models.**

Introduction

Automatic text classification is a fundamental problem in natural language processing, especially for organizing large-scale question–answer systems. Efficient categorization enables faster retrieval, improved user experience, and scalable support systems. The motivation of this project is to investigate how different text representation techniques and model architectures perform on a large multi-class dataset and to compare traditional machine learning approaches with deep learning–based methods. The dataset used in this project consists of labeled question–answer pairs provided in CSV format. Two datasets are used: a training dataset and a test dataset. Each record in the dataset contains a question, its corresponding answer, and an associated class label. The dataset consists of 93,333 samples of question–answer pairs, already split into training and testing sets. The training set was exclusively used for training and validation, while the test set was reserved for final evaluation. Each sample belongs to one of 10 distinct domain classes, such as “Business & Finance” and “Computer & Internet”. The corpus is sufficient for both traditional and neural models.

I. Dataset

The dataset consists of labeled question–answer pairs from different domains. Sample size is 93,333 against 10 classes, already split into training and testing sets. The training set was exclusively used for training and validation, while the test set was reserved for final evaluation. Each sample belongs to one of 10 distinct domain classes, such as “Business & Finance” and “Computer & Internet”. The corpus is sufficient for both traditional and neural models.

	QA Text	Class
0	<html> Question Title:\n What are the nam...	Science & Mathematics
1	<html> Question Title:\n can you tell me ...	Education & Reference
2	<html> Question Title:\n Suggestions or A...	Politics & Government
3	<html> Question Title:\n where can i find...	Entertainment & Music
4	<html> Question Title:\n will the first t...	Sports

Table 1: Sample Dataset head

II. Methodology

A. *Exploratory Data Analysis*

Data integrity and quality checks are used for the initial inspection of EDA. No duplication was identified, missing values were examined and dealt with if any null values or data issues arose, class distribution was checked for balance or imbalance across classification categories, text statistics examined text length distributions, and visualization was finished by making plots to comprehend data patterns.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

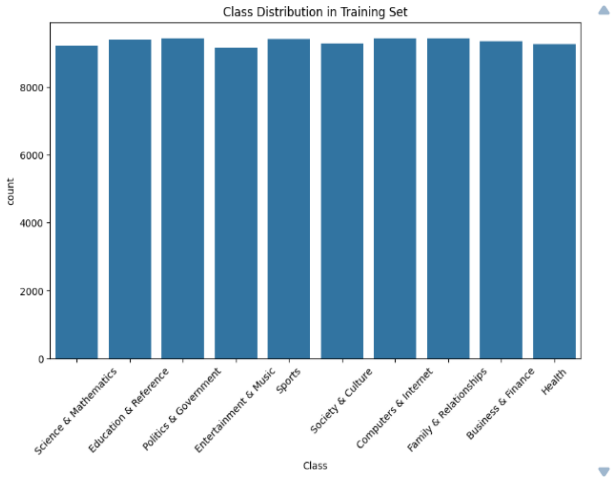

Fig 1: Class Distribution

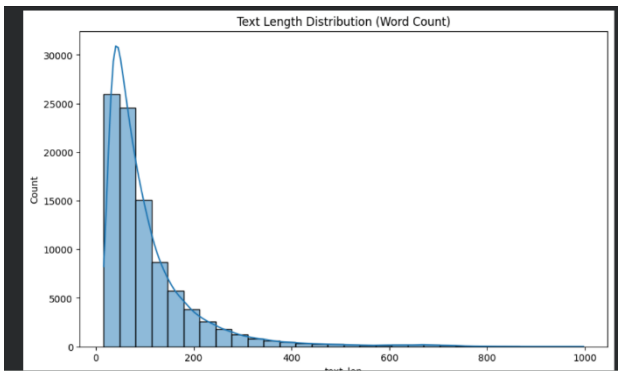

Fig 2: Text Length Distribution

B. *Preprocessing Techniques*

Text preprocessing and standardization applied to clean the raw data. In this portion different preprocessing approaches were explored but some cleaning was mandatory to all. Striping HTML tags and special characters, converting all texts to lowercase for consistency, tokenization, breaking text into individual tokens and removal of common unnecessary English words (stop words), lastly stemming/lemmatization basically converting words to their base forms to reduce vocabulary sparsity was applied as preprocessing techniques.

C. *Word Representation Methods*

Different embedding strategies are followed in this task. Firstly TF-IDF (Term Frequency- Inverse Document Frequency) which will convert text to numerical vectors based on word importance and capture term frequency with domain-level relevance [1][5]. Secondly, Skip-gram of 100 dimension (Word2Vec) applied which helps learn a rich, dense vector representations of words from context, capturing semantic relationship between words.

D. *Model Architectures*

First, For comparison purposes we started with a traditional Machine Learning architecture. Hyperparameter tuning, stemming, and lemmatization experiments were conducted across models.

- *Logistic regression*: A linear baseline model with TF-IDF features.

Then a Deep Model was introduced.

- *Deep Neural Network* [3]: A 2 layered NN model with dropout and TF-IDF features.

After that, Skip-gram was implemented with all 7 Neural Network models.

- *Deep Neural Network*: Dense layered network with skip-gram embeddings.
- *Recurrent Neural Network*: Basic sequential processing with recurrent connections.
- *Long Short-Term Memory* [2]: Captures long-range memories over the period of time.
- *Gated Recurrent Unit*: Similar to LSTM with fewer parameters.
- *Bidirectional Variants:* Bidirectional version of GRU LSTM and Simple RNN.

Early-Stopping and Model-Checkpoint were used to prevent overfitting and save the best models throughout the process. For each model, we first hyperparameter tuned them with the help of multiple trial and validation score. We first run a specific model for maximum 5 trials and then choose the best hyperparameter as our best model. Then we run the best hyperparameter tuned model with 150 epochs for the training of the model. Then we evaluate the model with the test dataset. We follow the same pipeline for all the seven neural network models.

III. Results

In this section, we present and discuss the classification performance of the various models developed to classification within the question-answer pair dataset. Each model was evaluated and assessed using precision, recall, F1-score, and accuracy. Confusion matrices for each final model are generated in the notebook using seaborn heatmaps.

A. *Model Performance Comparison*

- **Logistic Regression (TF-IDF):** Stemming-based model achieved approximately 65.65% accuracy and a macro F1-score of 0.65. Lemmatization-based model achieved approximately 65.18% accuracy and a macro F1-score of 0.65. Class-wise analysis and confusion matrices show that stemming reduced misclassification among closely related categories better.

```
Confusion Matrix:
[[2838  347  533  326  322  292  382  381  440  139]
 [ 281 4934  176  194   57   42   54  140   57   65]
 [ 442  180 2791  243  131  222  402  955  475  159]
 [ 251  231  268 3798  278  172  147  205  397  253]
 [ 237   64  162  330 4153  320  121   71  443   98]
 [ 157   32  174  109  284 4542   68  342  200   92]
 [ 422   47  312  113   89   74 4356  157  340   90]
 [ 164  102  495   93   29  310   87 4470  152   98]
 [ 263   65  538  384  536  248  388  250 3208  120]
 [ 153   60  134  194   60  127   86  153   85 4948]]
```

Fig 3: Confusion Matrix of Best logistic Regression (Stemming)

- **Deep Neural Network (TF-IDF):** Stemming resulted accuracy of 66.8% and Lemmatization resulted accuracy of 66.46%. Both pipelines were evaluated and compared using bar plots.

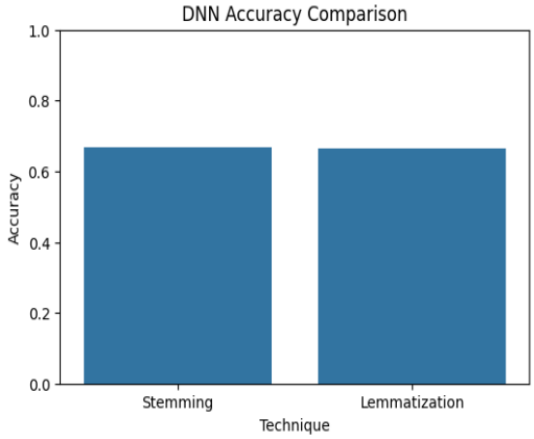

Fig 4: Comparison of DNN between different preprocessing technique

- **Deep Neural Network (Skip-gram):** - Achieved approximately 68% accuracy and 67% macro F1-score.

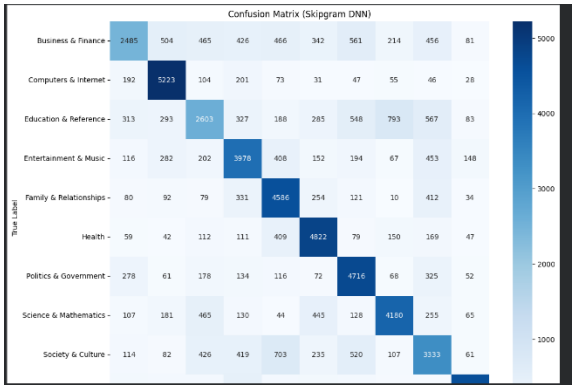

Fig 5: Confusion Matrix of DNN (skip-gram)

- **Simple-RNN (Skip-gram):** - Achieved approximately 66.9% accuracy and 66.3% macro F1-score with 160 RNN units, 0.1 dropout rate and activating dense layer (32 dense units and 0.1 dropout).

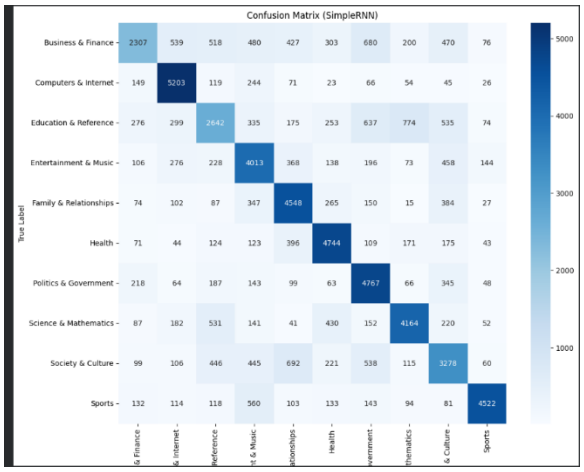

Fig 6: Confusion Matrix of Simple-RNN

- **GRU (Skip-gram):** - Achieved approximately 66.8% accuracy and 66.3% macro F1-score with hyperparameters at 64 GRU units 0.2 dropout rate, 0.001 learning rate and activating dense layer.

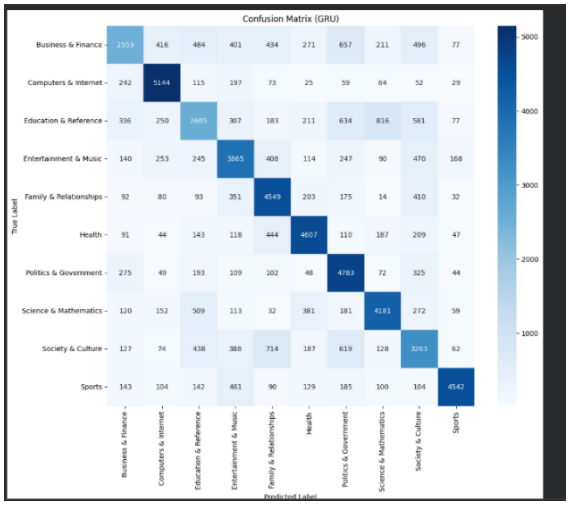

Fig 7: Confusion Matrix of GRU

- **LSTM (Skip-gram):** - Achieved approximately 66.8% accuracy and 66.7% macro F1-score with hyperparameters at 256 LSTM units, 0.3 dropout, 0.001 learning rate and deactivating dense layer.

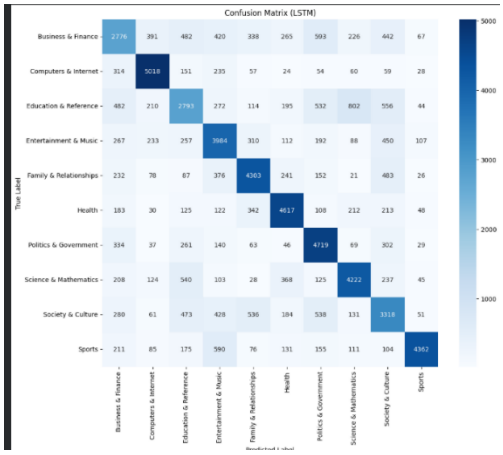

Fig 8: Confusion Matrix of LSTM

- **Bidirectional Simple-RNN (Skip-gram):** achieved 66.8% accuracy and 66.5% macro F1-score with hyperparameters of 160 RNN units, 0.2 dropout, 0.001 learning rate and activating dense layer (32 dense units and 0.1 dropout).

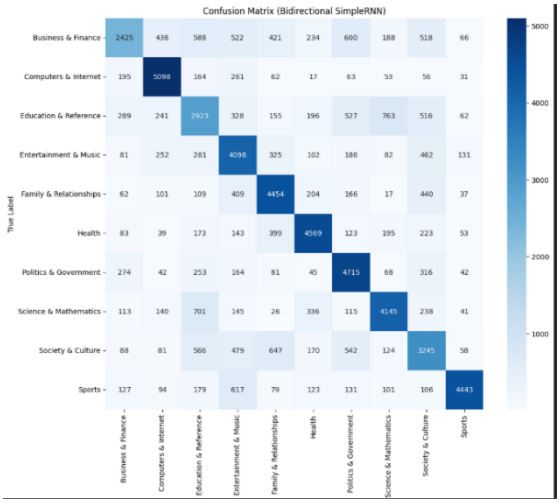

Fig 9: Confusion Matrix of Bidirectional Simple-RNN

- **Bidirectional GRU (Skip-gram):** achieved 67% accuracy and 66.5% macro F1-score with Hyperparameters set at 64 GRU units, 0.3 dropout, 0.001 learning rate and activating dense layer (64 dense units and dropout 0.2).

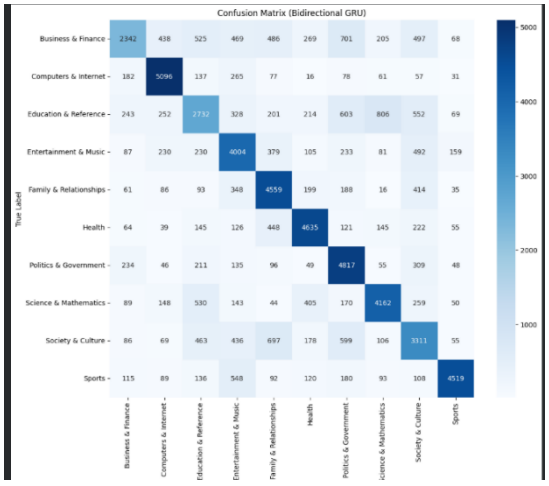

Fig 10: Confusion Matrix of Bidirectional GRU

- **Bidirectional LSTM (Skip-gram):** achieved 67.1% accuracy and 66.8% macro F1-score with Hyperparameters set at 256 LSTM units, 0.3 dropout, 0.001 learning rate and deactivating dense layer.

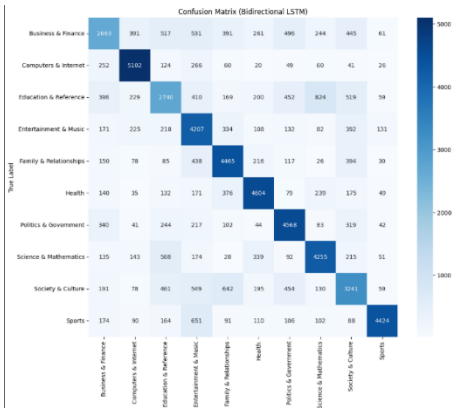

Fig 11: Confusion Matrix of Bidirectional LSTM

Here is the comparison of the accuracy and Macro F1 score of all the Neural Network Model:

MODEL	ACCURACY	MACRO-F1
DNN	0.6754	0.6696
SINPLERNN	0.6698	0.6637
GRU	0.6682	0.6638`
LSTM	0.6685	0.6673
BI-SIMPLERNN	0.6686	0.6659
BI–GRU	0.6696	0.6650
BI-LSTM	0.6712	0.6688

Table 2: Comparison between all the model

The DNN model is the best-performing model, achieving the highest test accuracy of 0.6754 and the highest macro F1-score of 0.6696. In contrast the GRU model is the worst-performing model by accuracy, with a test accuracy of 0.6682. The Simple-RNN model is the worst-performing model by F1-score, with a macro F1-score of 0.6637.

IV. Conclusion

A. *Key Findings*

*The DNN model outperformed all other models, achieving the highest test accuracy of 0.6754 and the highest macro F1-score of 0.6696.

* The GRU model exhibited the lowest test accuracy and the Simple-RNN model recorded the lowest macro F1-score.

*Stemming-based preprocessing slightly outperformed lemmatization.

* Baseline TF-IDF + Logistic Regression gives ~65.65% accuracy which is a strong metric compared to other modern models .

Overall, the performance metrics across all models were quite close, whether its accuracy or F1-score, indicating a narrow performance spread.

B. *Limitations*

The proposed system primarily relies on TF-IDF-based feature representation, which captures word frequency but fails to model deeper semantic and contextual relationships. As a result, the model may struggle with complex or paraphrased question–answer pairs. Although word embedding techniques such as Skip-gram and neural network–based models can better capture semantic information, they were not fully utilized due to dataset size and computational constraints. Additionally, class imbalance and limited training data may affect performance on minority classes. Finally, evaluation using a single train–test split may not fully reflect the model's generalization ability in real-world scenarios.

C. *Insights and Future Work*

The project implements end-to-end pipelines: preprocessing, TF-IDF baselines, Word2Vec Skip-gram embeddings, tuned DNNs and RNNs. Baseline TF-IDF + Logistic Regression (65% accuracy); embedding-based neural models are competitive and can be improved with additional tuning or richer sequence representations.

This study demonstrates the effectiveness of combining advanced preprocessing, feature engineering, and deep learning for multi-class text classification. Future work includes exploring transformer-based models (e.g., BERT[4]), larger pre-trained embeddings, and advanced data augmentation or ensemble methods.

REFERENCES

[1] T. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.

[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.

[3] F. Chollet, "Deep Learning with Python," Manning, 2017.

[4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.

[5] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv.